MG Installation and Training Examples

© Brian Maxwell and Team, University of Ottawa

June 5, 2026

Contents

1 Installation **2**
1.1 YouTube Videos . . . 2
1.2 Packages needed for installation on Ubuntu . . . 2
1.3 Installation using bash terminal in Ubuntu . . . 3
1.4 Installation using csh/tcsh terminal . . . 4
1.5 Installation using Mac . . . 4
1.6 Accessing and Installation on HPC . . . 4
1.6.1 Accessing HPCs . . . 4
1.6.2 Installation on HPC . . . 5
1.6.3 Set up ssh keys for bitbucket . . . 6
1.6.4 Some useful commands to use on HPC systems . . . 6

2 The calorically perfect gas model (MG_G) **6**
2.1 Governing Euler equations of motion . . . 6
2.2 The basic script . . . 7
2.3 Training examples to try . . . 7

3 The one-step combustion model (code with linear eddy model) (MG_LEM) **10**
3.1 Governing Euler equations of motion . . . 10
3.2 The source term . . . 10
3.3 The basic script . . . 11
3.4 Training examples to try . . . 12

4 Detailed chemistry code (MG_CTI) **22**
4.1 Governing Euler equations of motion . . . 22
4.2 The source terms . . . 22
4.3 The basic script . . . 24
4.4 Training examples to try . . . 25

5 CFD workflow - Fundamentals **26**
5.1 Stage 1: Pre-processing 26
5.2 Stage 2: Solving 27
5.3 Stage 3: Post-processing 28

6 MG Code Files **29**
6.1 1. Data structures and definitions 29
6.2 2. Initialization, setup, execution, and memory allocation 29
6.3 3. Numerical solvers and integration 29
6.4 4. Mesh management 30
6.5 5. Parallelization and I/O 30
6.6 6. User customization 30
6.7 How are they linked together? 30

1 Installation

1.1 YouTube Videos

As a reference, some useful videos covering the MG installation process and basic navigation can be found at the following links:

- MG download using git and install using Bash:
  `https://youtu.be/UiuRLH5mtxE`
- MG Navigation:
  `https://youtu.be/KGIHW5S3C7k`

1.2 Packages needed for installation on Ubuntu

Note: MG should be installed as a local user on your machine and not as **root**!

First, before installing new software, it's always a good idea to update the packages on your machine:

```
sudo apt update
sudo apt upgrade
```

Install git and the open ssh client for downloading the MG source code and examples. Vim is a good terminal based text editor that you should also download:

```
sudo apt install git
sudo apt install openssh-client
sudo apt install vim
```

At this point, if using Ubuntu, we need to configure vim, otherwise the keyboard may not function properly when using the program. To configure vim type the following:

```
cd ~
touch .vimrc
echo "set nocompatible" >> ~/.vimrc
```

Then, compilers and tools that are needed:

```
sudo apt install tcsh make gcc g++ gfortran openmpi-bin
```

Aside from the usual standard packages included with Linux, you will likely need to install development libraries for open mpi, x11 graphics, readline, gsl, and ncurses. In Ubuntu, these can be installed with:

```
sudo apt install libopenmpi-dev
sudo apt install libx11-dev
sudo apt install libreadline-dev
sudo apt install libgsl-dev
sudo apt install libncurses5-dev libncursesw5-dev
```

1.3 Installation using bash terminal in Ubuntu

Installation should be rather straightforward, compared to larger and more complex CFD codes, but requires a degree of competence using and navigating a Linux terminal environment. In your terminal, your fist step should be to download the mg repository to a folder, preferably located in your main home folder. However, you can in theory download it anywhere you want, and even have multiple versions as long as the system knows where to find it. To download mg using git, first navigate to your home folder:

```
cd $HOME
```

Now download the software (access to the bitbucket is required - you may have to setup SSH keys to access):

```
git clone git@bitbucket.org:bmaxw005/mg.git
```

Note, if you are unable to download via the command line, you may need to configure the appropriate SSH keys on your system, see https://support.atlassian.com/bitbucket-cloud/docs/set-up-personal-ssh-keys-on-linux/ The next step, if using bash, is to setup your environment. First we need to add two lines to your .bashrc file (located in your `$HOME` folder). The lines that need to be added are:

```
export mg=$HOME/mg
source $mg/_bashrc
```

You can add these lines to .bashrc using vim, or another text editor, or even from the command line itself. For example, in the terminal you can type the following to add text directly to the file without using any text editor:

```
echo "export mg=~/mg" >> $HOME/.bashrc
echo "source $mg/_bashrc" >> $HOME/.bashrc
```

Next you can reload your .bashrc file by opening a new terminal window or typing

```
source $HOME/.bashrc
```

Next, navigate to the mg folder:

```
cd $mg
```

Here you'll want to edit the _bashrc file to suit your needs. There are different options if you wish to compile using single or double prescision, or if you wish to compile using optimized settings or debugger flags. Any time you make a change to this file, it is adised to clear any precompiled version of MG using the `Clearall` command (located in the `$mg` folder). To compile the main libraries of MG, type:

```
Createlib
```

Different versions of MG can now be compiled. For example, to compile the MG_G version, navigate to the corresponding folder:

```
cd $mg/mg_g
```

Now to compile the code you have two options. To compile on a shared memory machine, using built in libraries for parallelization, you can compile with

```
makemg smp
```

To compile using mpi libraries, which work well for both shared memory and distributed memory machines, compile with

```
makemg mpi
```

At successful build, the mg_g binary should appear in the bin subfolder, and be accessible to your system from anywhere. To test it, type `mg_g` from anywhere on your computer. To clear only this version of MG, without affecting other MG programs, you can type `Clear` from the mg_g folder. Other versions of MG are installed in the same manner.

1.4 Installation using csh/tcsh terminal

1.5 Installation using Mac

1.6 Accessing and Installation on HPC

1.6.1 Accessing HPCs

To access a Digital Alliance of Canada system, you will first need to generate an SSH key on your local machine. In your ubuntu terminal, go to your home directory and look for ssh keys in

```
cd .ssh
```

which contains both private SSH keys and a public key (key.pub). If you do not have any keys in this folder, you need to generate your key using

```
ssh-keygen -t rsa -b 4096
```

By default, you can skip the prompt asking "which file to save your key", and its passphrase. Next, type

```
cat id_rsa.pub
```

to print your public key to your ubuntu prompt, which looks like

```
ssh-rsa LongSeriesOfNumbersAndLetters123=AndSomeSymbols
UserName@LocaMachine
```

Copy this entire string. Then go onto the alliance webportal, ccdb.alliancecan.ca → My Account → SSH Keys, and paste this key into the appropriate box. Once this is done, you can log into a CCDB machine from your home ubuntu terminal using the command

```
ssh username@rorqual.alliancecan.ca
```

To make your life easier, you can add your username as an environment variable in your local .bashrc as

```
export rorqual=username@rorqual.alliancecan.ca
export narval=username@narval.alliancecan.ca
```

and re-sourcing your .bashrc. Following this, you can access these machines using

```
ssh $rorqual
```

Transferring a file from your local machine to or from these machines can be done using rsync (preferred) or scp, as

```
rsync -avz (LOCAL PATH) $rorqual:~/CLUSTERPATH
rsync -avz $rorqual:~/CLUSTERPATH (LOCALPATH)
```

The flags in this command are

```
-a (Archive) to preserve permissions, timestamps, etc.,
-v (Verbose) to show progress,
-z (Compress) to compress the transfer,
--dry-run allows you to preview the transfer
```

The `:~` following the machine name indicates your home directory on rorqual. The colon following the host name indicates a remote machine.

1.6.2 Installation on HPC

On the Digital Alliance of Canada systems (Rorqual, Nibi, Narval, Fir), you will likely only need to load gsl prior to compiling and running. Add the following line to your .bashrc:

```
module load gsl
```

On the Trillium system, you will need to load the following:

```
module load gcc/12.3
module load StdEnv/2023
module load openmpi
module load gsl
```

Compiling and running MG then follows the usual instructions for bash, see section section 1.3. Note: **DO NOT USE SUDO ON THESE MACHINES.**

1.6.3 Set up ssh keys for bitbucket

1.6.4 Some useful commands to use on HPC systems

Add some useful commands etc

2 The calorically perfect gas model (MG_G)

2.1 Governing Euler equations of motion

Conservation of mass:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho u) = 0 \tag{1}$$

Conservation of momentum:

$$\frac{\partial (\rho \boldsymbol{u})}{\partial t} + \nabla \cdot (\rho u \boldsymbol{u}) + \nabla p = 0 \tag{2}$$

Conservation of total energy (internal+kinetic):

$$\frac{\partial (\rho e)}{\partial t} + \nabla \cdot \Big((\rho e + p) \boldsymbol{u} \Big) = 0 \tag{3}$$

Conserved scalar:

$$\frac{\partial (\rho Y)}{\partial t} + \nabla \cdot (\rho \boldsymbol{u} Y) = 0 \tag{4}$$

where the calorically perfect gas equations of state are given by

$$e = \frac{p/\rho}{(\gamma - 1)} + \frac{1}{2} \boldsymbol{u} \cdot \boldsymbol{u} \tag{5}$$

$$\rho T = p \tag{6}$$

The various parameters and variables are given in non-dimensional form according to

$$\rho = \frac{\hat{\rho}}{\hat{\rho_o}}, \qquad u = \frac{\hat{u}}{\hat{c_o}}, \qquad p = \frac{\hat{p}}{\hat{\rho_o} \hat{c_o}^2} = \frac{\hat{p}}{\gamma \hat{p_o}}, \qquad T = \frac{\hat{T}}{\gamma \hat{T_o}}, \qquad x = \frac{\hat{x}}{\hat{L}}, \qquad t = \frac{\hat{t}}{\hat{L}/\hat{c_o}},$$

2.2 The basic script

As a first example, we will look at a shock wave encountering and diffracting around a circular object. The following basic can be referenced (`obstacle.dat`)

```
! Solution tolerance and refinement criteria for AMR
rtol 1E-3
refcrit 0.1
derefcrit 0.05
remhole
! Value of gamma=cp/cv
gamma 1.4
! turn on internal geometry
solidbody
! Inviscid flow
noviscous
! Load the initial conditions and initial AMR refinement
slow
diffbad 20
START "obstacle test"
   LEVELS 5
   XYZ 2
      X 20 -2.0 6.0 FREE
      Y 10 -2.0 2.0 FREE
      SCALARS 1 ADVECT 1
   CASE OBSTACLE
END
diffbad 5
```

At present, `obstacle.dat` is located in the mg_g folder.

2.3 Training examples to try

1. Run the above script manually, by navigating to the mg_g folder and launching the `mg_g` command. At the MG_G> prompt type the following sequence of commands:

   ```
   $read obstacle
   $read x
   plot rho p
   march 40
   er p
   g
   ```

   Here you should see the gasdynamics as a shock wave diffracts around an obstacle. You should also see the grid topology that is generated. `$read obstacle` is the command that loads obstacle.dat in the local folder, `$read x` will load a file called x.da t, which also has to be located in the

local folder. This file will handle opening of the graphics window. `plot` tells the interpreter what variable to plot, while `march 40` advances the simulation by 40 time steps on the coarsest grid. `p` is used to plot or replot to the screen, and `g` shows the grid topology. We can view how many grid points on each level there are with the `stats` command. You can also view stored custom variables in memory with the `users` command. For example, the value of γ is reported here. You can also see other useful information using the `show` command. For example, you can find the current time of the simulation here. Now let's plot something else, like the temperature by issuing the following

```
er
plot te p
```

Note `er` erases the canvas, and we can plot different primitive variables using the plot command. For example, you can also plot u and v velocities, or pressure by typing `plot u1`, `plot u2`, or `plot pg`. The full list of what can be plotted is found in the `guide.html` file found in the docs subfolder of MG. We can quit the MG program with the `q` command

2. Toggle the cylindrical object on and off by commenting out the `solidbody` command in the `obstacle.dat` file. The line can be commented out by adding a ! at the beginning of the line. Repeat ex. 1 by comparing the flow fields with and without an obstacle.

3. Let's change the size of the obstacle from Ex. 1 by modifying the source code in the `usrqis` function. Look for the `OBSTACLE` case, and the part that defines a circle of radius 1. Change it to define a circle of radius 0.5. Recompile mg_g and rerun Ex. 1.

4. It is much more convenient to define the cylinder size as a variable that can be changed on the fly, rather than have to recompile the code every time. Lets add a command called "`RADIUS`" that will store the given command argument into a global variable, called `radius`. The first step is to add a `float` type variable called `radius` to the `user.h` file in the mg_g folder. Then in the `usrcmd` function in `usr.c` you'll have to add some new lines to accommodate reading a command and populating the new variable. Have a look at what is done for the "`ABSTOL`" command and implement a new command to populate your radius variable. You will also have to incorporate how the new `radius` variable is saved to or read from save state files by making changes to the `usrwr` and `usrrd` functions, also in `usr.c`. Then, you should give a default value for the `radius` variable in the `usrdef` function. It is also good practice to output the value of `radius` in the `usrprt` function, which is invoked by issuing the `users` command. Once the changes are made, recompile mg_g. Finally, modify the `OBSTACLE` case in the `usrqis` function to specify the cyclinder size according to the value of `radius`. If your changes are successful during

a recompile, add the command `radius 0.5` to `obstacle.dat` before the `START` block and rerun Ex. 1.

5. We can add Lagrangian particles to the simulation by adding the `particle` command to the script, before the `START` block. Do this and re-run the simulation. Specific rules for where the particle is created for the `OBSTACLE` case is found in the `usrpartpop` function of the `usr.c` file. At present 7 particles are created. Can you create 14 particles instead, and have them all equally spaced?

6. At this point you should try running example 1 again, but with different resolutions. You should look at the `START` block and try different number of refinement levels, and try different number of base grid points, and so on. Get a good feel for how the START block is setup and what it does to the resolution.

7. Add source term for combustion (a one-step model). This can be achieved by first making sure a scalar is present in the `obstacle.dat` file. Actually maybe we will create a new script from scratch... Then, we should treat this scalar as a progress variable of the reaction, lets call it λ, which will go from having a value of 0 in pure premixed reactants to a value of 1 in the products. In this sense, the mas fraction of reactant is given by $(1-\lambda)$, while the mass fraction of the products is λ. Thus, equation (4) should be replaced by

$$\frac{\partial(\rho\lambda)}{\partial t} + \nabla\cdot(\rho\boldsymbol{u}\lambda) = \dot{\omega} \tag{7}$$

where

$$\dot{\omega} = \rho A(1-\lambda)\exp\left(-E_a/T\right) \tag{8}$$

Finally, to reflect the contribution to the energy balance, the equation of state, equation (5) should also be modified:

$$e = \frac{p/\rho}{(\gamma-1)} + \frac{1}{2}\boldsymbol{u}\cdot\boldsymbol{u} - \lambda Q \tag{9}$$

where E_a, A, and Q, and model parameters, that should be brought in by creating new commands. Default values of $E_a = 45$, $A = 8000$, $Q = 50$ (with $\gamma = 1.17$) would roughly corresponds to $CH_4 + 2O_2$ combustion at $\hat{T} = 300$ K amd $\hat{p} = 3500$ Pa. It is your task to find a way to incorporate these changes into your mg_g code, by modifying the `usr.c` file in several places.

3 The one-step combustion model (code with linear eddy model) (MG_LEM)

3.1 Governing Euler equations of motion

Conservation of mass:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho u) = 0 \tag{10}$$

Conservation of momentum:

$$\frac{\partial (\rho \boldsymbol{u})}{\partial t} + \nabla \cdot (\rho u \boldsymbol{u}) + \nabla p = 0 \tag{11}$$

Conservation of sensible+kinetic energy:

$$\frac{\partial (\rho e)}{\partial t} + \nabla \cdot \Big((\rho e + p) \boldsymbol{u} \Big) = -Q\dot{\omega} \tag{12}$$

Mass of reactant:

$$\frac{\partial (\rho Y)}{\partial t} + \nabla \cdot (\rho \boldsymbol{u} Y) = \dot{\omega} \tag{13}$$

where the calorically perfect gas equations of state are given by

$$e = \frac{p/\rho}{(\gamma - 1)} + \frac{1}{2} \boldsymbol{u} \cdot \boldsymbol{u} \tag{14}$$

$$\rho T = p \tag{15}$$

The various parameters and variables are given in non-dimensional form according to

$$\rho = \frac{\hat{\rho}}{\hat{\rho_o}}, \qquad u = \frac{\hat{u}}{\hat{c_o}}, \qquad p = \frac{\hat{p}}{\hat{\rho_o}\hat{c_o}^2} = \frac{\hat{p}}{\gamma \hat{p_o}}, \qquad T = \frac{\hat{T}}{\gamma \hat{T_o}}, \qquad x = \frac{\hat{x}}{\hat{L}}, \qquad t = \frac{\hat{t}}{\hat{L}/\hat{c_o}},$$

3.2 The source term

The most basic combustion model included with MG LEM, when the `reactive` command is invoked, is a one-step reaction model that takes the form:

$$\dot{\omega} = -\rho^n T^m A Y^\beta e^{(-E_a/T - E^*(1-Y))} \tag{16}$$

By default, the coefficients (i.e. reaction orders) $n = 1$, $m = 0$, $\beta = 1$, and $E^* = 0$. To adjust these state variable dependencies, the user can invoke the `dorder`, `Torder`, `rxnorder`, and `Ecb` commands followed by the respective values. Thus, by default:

$$\dot{\omega} = -\rho A Y e^{(-E_a/T)} \tag{17}$$

The other relevant combustion parameters (E_a, A, and Q) are controlled by the `Ea`, `A`, and `Qrxn` commands. For this example, $E_a = 45$, $A = 8000$, $Q = 50$ and $\gamma = 1.17$, which roughly corresponds to $CH_4 + 2O_2$ combustion at $\hat{T} = 300$ K amd $\hat{p} = 3500$ Pa. These parameters are normalized by

$$E_a = \frac{\hat{E}_a}{\hat{c_o}^2}, \qquad Q = \frac{\hat{Q}}{\hat{c_o}^2}, \qquad A = \frac{\hat{A}}{\hat{c_o}/(\hat{L}\hat{\rho_o}^{n-1})} \tag{18}$$

In this case, the length scale for normalization will correspond to $\hat{L} = 10$ mm.

3.3 The basic script

To invoke the one-step combustion model in MG, the following basic script that models a detonation wave in methane-oxygen can be referenced (`detonation2D.dat`)

```
! tolerances
rtol 1E-4
abstol 1E-4

! other grid refinement parameters
refcrit 0.1
derefcrit 0.05

! solution tolerance in the ZND solver
ztol 1E-2

! solver settings
reactive
implicit     ! default solver (implicit, explicit, rk4, gsl)
godunov      ! default solver (godunov, hll, roe)
vanalbada    ! default limiter (vanalbada, vanleer, minmod, superbee, mc)

! model parameters
gamma 1.17
reactionorder 1
Ea 45
A 8000
Qrxn 50

! slightly overdrive to overcome startup errors
overdrive 1.1

! track triple point locations using numerical open shutter
! note:  alternate sootfoil command requires Navier-Stokes mode
openshutter

! load 2D detonation case
```

```
! temporarily increase diffbad to initialize grid
slow
diffbad 20
! each nondimensional unit is 10 mm
START "2D CH4-O2 Detonation"
    LEVELS 4
    XYZ 2
        X 250 -20.0 980.0 WALL-FREE
        Y 5 0.0 20.0 SYMMETRIC
        SCALARS 1 ADVECT 1
    CASE ZND2D
END
diffbad 5
```

By default, everything has been setup for you in the `usr.c` file for the `ZND2D` case. You can search for "ZND2D" in `usr.c` to see where specific custom directives for this case are included. This case calculates and sets up a ZND profile located at $x = 0$, overdriven slightly ($f = (D^2/D_{cj}^2) = 1.1$) to overcome startup errors at the beginning. A small zone of perturbations to the density field are located between $0 < x < 4$.

For the example below, you can download the run scripts from the following bitbucket repository:

```
git clone git@bitbucket.org:bmaxw005/mg_trainingexample1.git
```

3.4 Training examples to try

1. Run the script for different resolutions, by increasing the number of AMR levels. Each AMR level effectively reduces the grid spacing in each direction by half. Compare (plot) the run times and open-shutter images while keeping the number of CPUs constant. On the plot, make note of what coarsest resolution is required to produce *cells*. The `time.sh` script can be used to add up the simulation times output in the `mgout` file generated. Report the resolutions both non-dimensionally, and in mm. Also, in a separate plot, report the % of computational cells that exist on the coarsest grid levels, G0 + G1, as the detonation wave approaches the end of the channel. The statistics of the computational cell distribution by grid level can be output with the `stats` command. To generate open-shutter images, the `openshutter.sh` script can be used as a guide.

   Samaneh's note:

   - nondimensionalization is done using the induction length of CJ detonation = 10mm
   - finest grid is chosen in terms of resolving the cellular structure of the detonation wave.
   - in a fine grid, almost half of the grids in the domain contain the smallest cell size
   - grid levels number 0 and 1 remain the same for all the levels and throughout the whole computational domain.

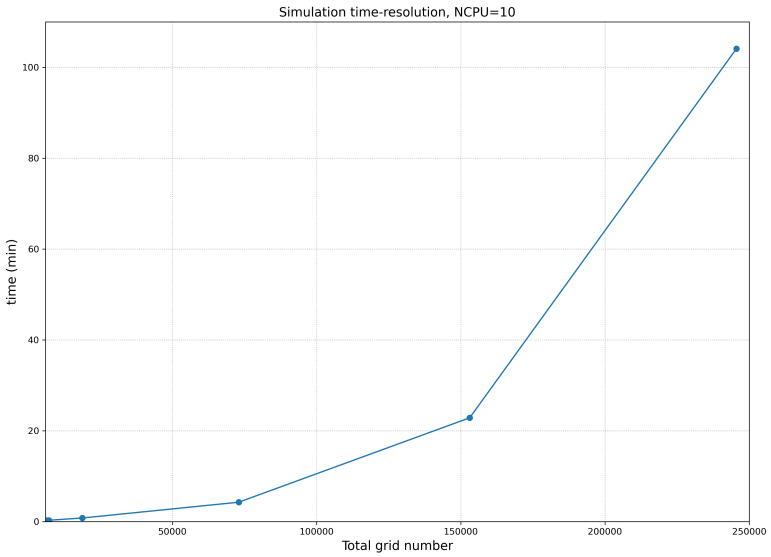
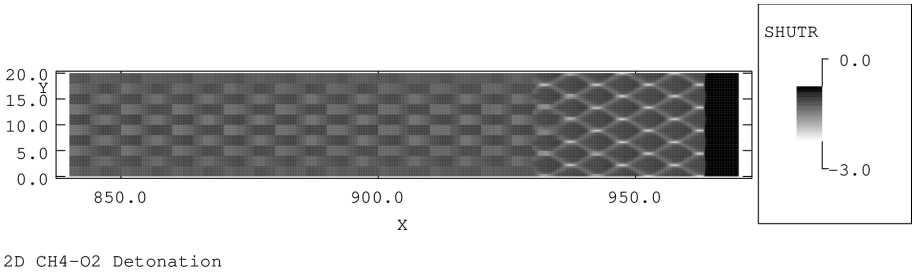


Figure 1: Simulation time-total grid number

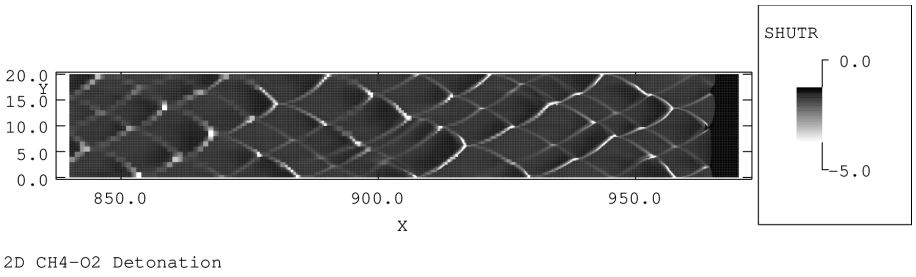
- comparing the grid resolution at a specified time step, with enhancing the grid levels(let's say from 3 to 7), the grid level of 3,4,5,6, all increase. Put differently, for a finer resolution, each cell size($4 * 0.5^2$, $4 * 0.5^3$,...) increase in number throughout the domain.

2. For one of the lowest resolution required to produce cells, start by running the script for 1 CPU and record the time needed for the simulation. Repeat the task over and over by doubling the number of CPUs. Thus, record and plot the simulation run time vs. n_{cpu}. Compare the cellular structure (i.e. numerical open shutter images) for each simulation. By analysing the `mgout` file generated and run times, estimate roughly how many cells per CPU is optimal.

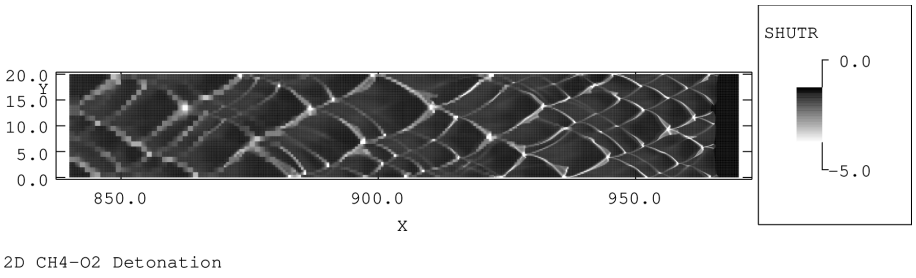
3. For an intermediate resolution (not too high, not too low), and pick a reasonable number of CPUs to use such that computations can be done in a reasonable time. Now, modify the main solution tolerances and grid refinement parameters, `rtol`, `abstol`, `diffbad`, `refcrit`, and `derefcrit`. How do changes in each of these parameters change the grid topology and the resulting open-shutter image? Explain what each of these parameters does. *Hint:* Try to decipher how these parameters control the refinement in the `usrqsol` function in `usr.c`. *Note:* `rtol` and `abstol` control both the AMR and newton iteration in the integration of the source term.



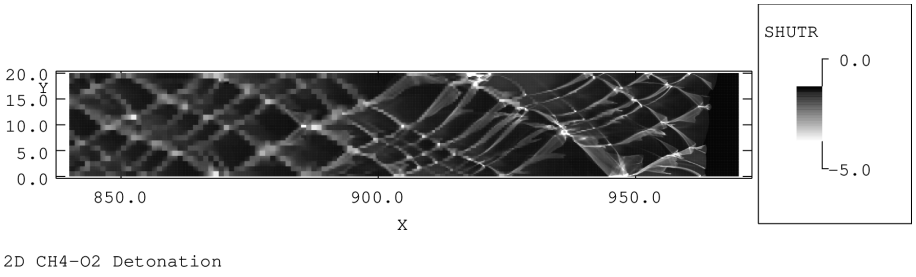
(a)



(b)



(c)



(d)

Figure 2: AMR comparision (a) finest resolution: 0.25 unit, 2.5 mm, (b) finest resolution: 0.125 unit, 1.25 mm (c) finest resolution: 0.0625 unit, 0.625 mm (d) finest resolution: 0.03125 unit, 0.3125 mm

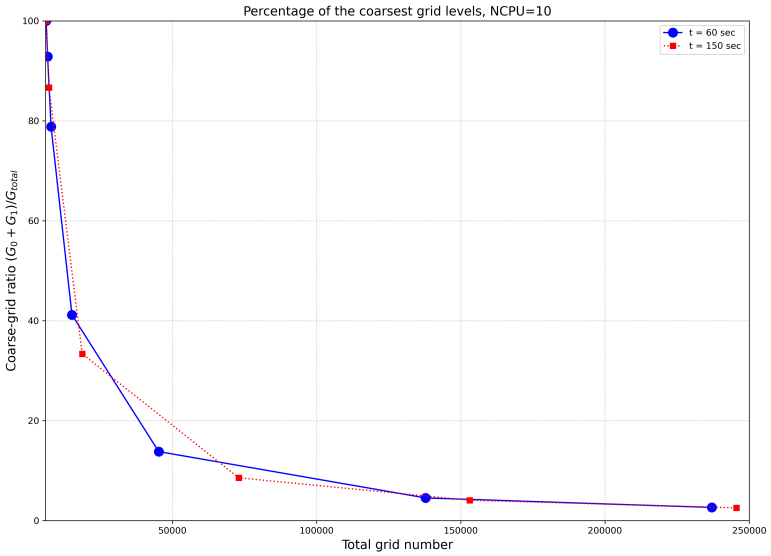


Figure 3: Percentage of coarse_grid_ratio $(G0 + G1)/Gtotal$ against total grid number

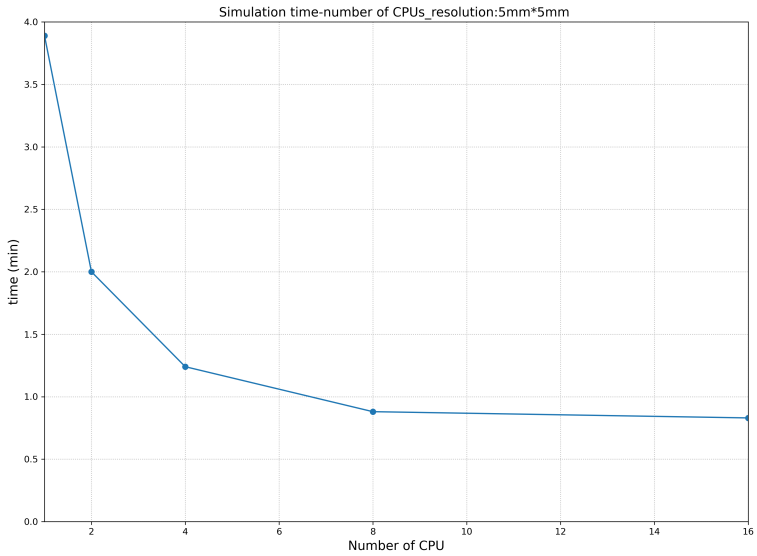


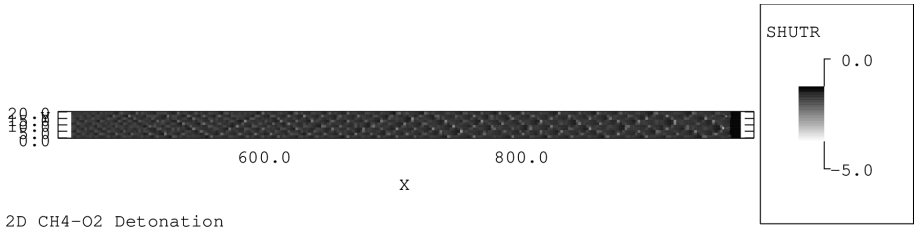
Figure 4: CPU-time for the grid resolution of 0.25*0.25

4. Pick a reasonable number of CPUs to use, satisfactory grid refinement strategy, and resolution from the previous tasks. Now vary independently the combustion parameters E_a, A, Q, and also the ratio of specific heats, γ. Explain how each of these physical parameters influences the cellular structures (i.e. cells) and detonation speed in terms of its Mach number and in m/s. To measure the wave speed as a function of distance, you can use, and modify as needed, the `post.sh` and `getspeed.sh` scripts. You can use Cantera to estimate the speed of sound in the reactants, $\hat{c}_0$, to work out the dimensional wave speed.

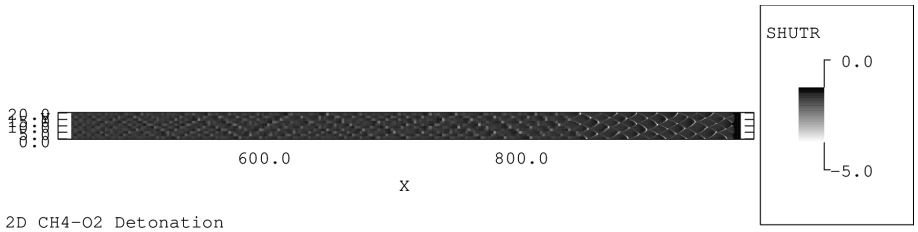
5. For $2H_2 + O_2$ combustion at $\hat{T} = 300$ K and $\hat{p} = 3500$ Pa, set the following: $E_a = 18$, $A = 50$, $Q = 15$ and $\gamma = 1.32$. In this case, the length scale for normalization will correspond to $\hat{L} = 2$ mm. For a fine resolution (within reason), compare the open-shutter image obtained with that corresponding to methane–oxygen combustion. What is different and why?

   Samaneh's note:

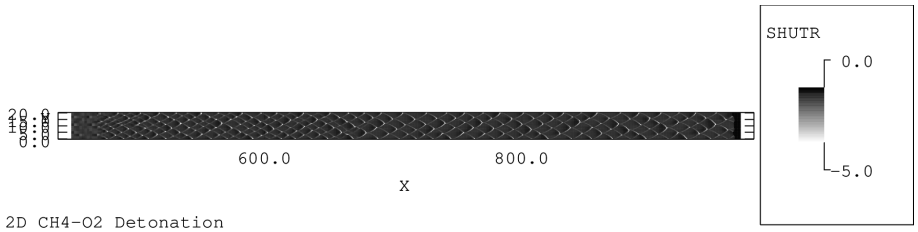
   - Use Cantera and SDToolbox to calculate $\hat{D_{CJ}}$ and $\hat{c}$, which gives us: 2655.4451 m/s anf 539.4808 m/s, respectively.
   - $D_{CJ} = \frac{\hat{D_{CJ}}}{\hat{c}} = 4.9217$
   - total time for the wave to reach the end of the channel: 1000 units / $D_{CJ} = 677$
   - divide 677 by 0.3 to find the total dimensionless time steps as an input in runsim.sh



(a)



(b)

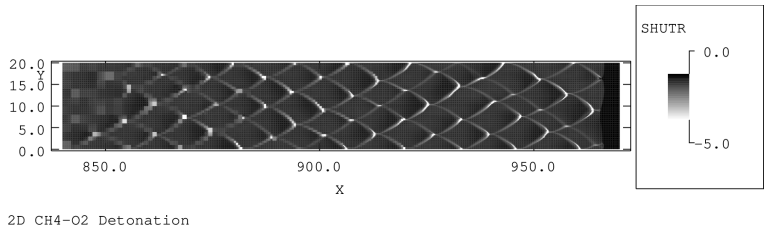


(c)

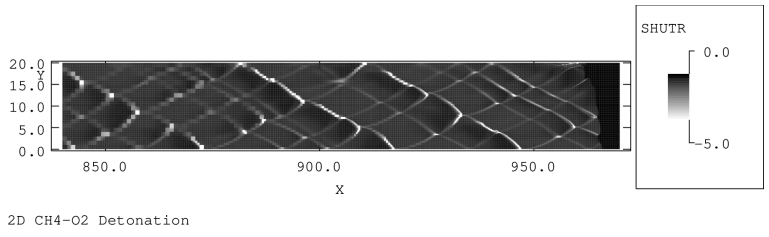


(d)

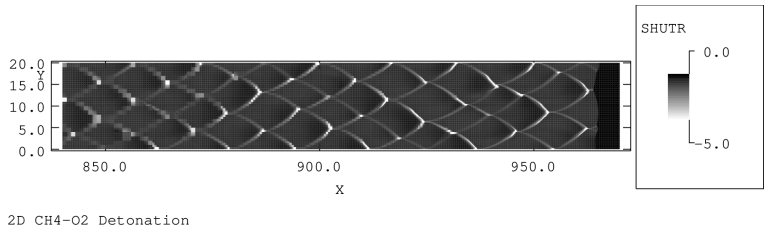
Figure 5: Comparing **rtol** values while CPU: 10, resolution: 0.125 units, abstol = 1e-4, refcrit= 0.1, derefcrit= 0.05, diffbad = 20, (a) 1e-2 (b) 1e-4 (c) 1e-5 (d) 1e-6



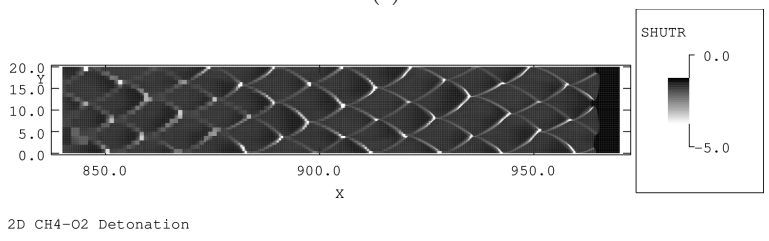
(a)



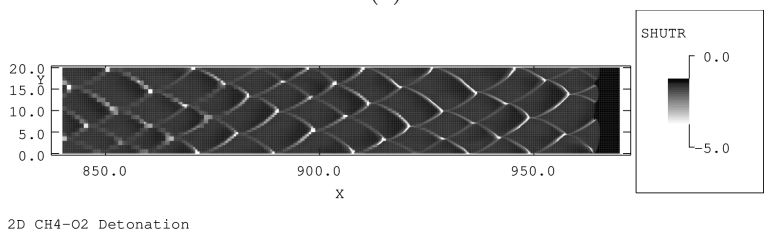
(b)



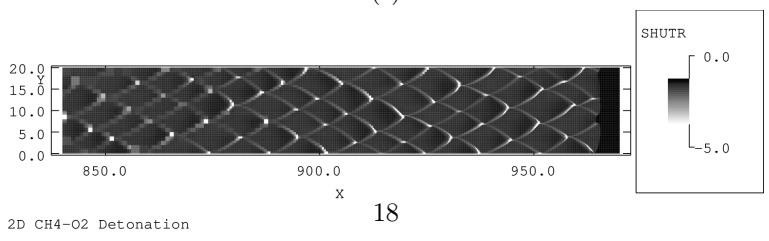
(c)



(d)



(e)



(f)

Figure 6: Comparing **abstol** values while CPU: 10, resolution: 0.125 units, rtol = 1e-4, refcrit= 0.1, derefcrit= 0.05, diffbad = 20, (a) 1 (b) 1e-2 (c) 1e-3 (d) 1e-4, (e) 1e-5, (f) 1e-6

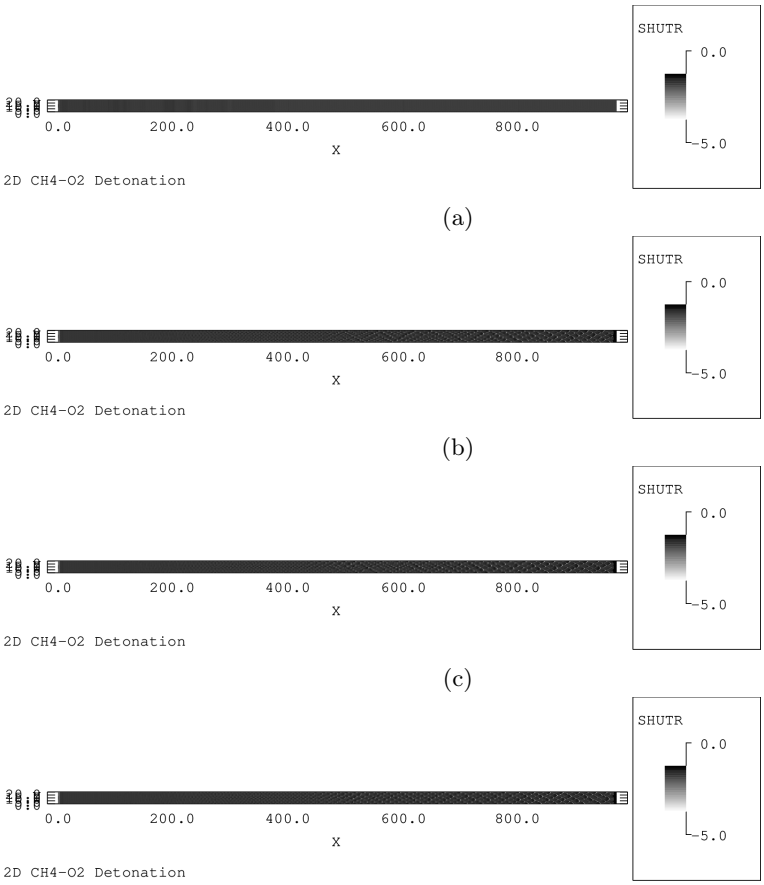


Figure 7: Comparing **refcrit** and **derefcrit** values while CPU: 10, resolution: 0.125 units, rtol = 1e-4, absto = 1e-4, diffbad = 20, (a) refcrit= 2, derefcrit= 1 (b) refcrit= 0.1, derefcrit= 0.05 (c) refcrit= 0.05, derefcrit= 0.025 (d) refcrit= 0.025, derefcrit= 0.01

- actual length is: $1000 * 2mm = 2m$

6. By default, or by issuing the `implicit` command, Eq. (16) is solved using a hard-coded and efficient Backward Euler (implicit) method scheme with variable time-stepping. The source term itself is solved in the `usrsaf` function, located in `usr.c`. Alternatively, Eq. (16) can be solved using a Forward Euler (explicit) scheme by invoking the `explicit` command. Runge-Kutta integration is also available by issuing the `rk4` command. The variable time-stepping may be toggled on/off using the `variable` and `novariable` commands. The minimum time step is controlled by the `htol` command followed by the value. Finally, the gsl ODE integrator may also be used by issuing the `gsl` command. For example to use Runge-Kutta integration with a non-variable minimum time step equal to $\Delta h = 0.0001\Delta t$ the following should be issued:

```
reactive
rk4
novariable
htol 0.0001
```

To use the gsl ODE integrator:

```
reactive
gsl
```

For $E_a = 45$, $A = 8000$, $Q = 50$, and $\gamma = 1.17$, run the code using each of these four solvers and compare the simulation times, and the resulting cellular structures (open-shutter images).

7. By default, MG_LEM uses the exact Godunov solver to solve the hyperbolic terms in the governing Euler equations. However, Roe and HLL solvers are also available by issuing the `roe` and `hll` commands, respectively. One can revert back to the Godunov solver by issuing the `godunov` command. These different solvers can be found in the `mg.c` file. For $E_a = 45$, $A = 8000$, $Q = 50$, and $\gamma = 1.17$, run the code using each of these three solvers and compare the simulation times, and the resulting open-shutter images.

8. A variety slope limiters are available for the 2nd order integration of the hyperbolic terms, see `funcs.h` in the `src` subfolder of MG. These are universal and apply to all versions of MG. By default, the Van Albada 1 slope limiter is used. Other available slope limiters are the Van Leer, Minmod, Superbee, and MC. These can each be invoked by issuing the `vanalbada`, `vanleer`, `minmod`, `superbee`, and `mc` commands, respectively. For $E_a = 45$, $A = 8000$, $Q = 50$, and $\gamma = 1.17$, and using the exact godunov solver, run the code using each of these five slope limiters and compare the simulation times, and the resulting open-shutter images.

9. Run the `sod.dat` script included with the MG_LEM program, which executes the SOD case and marches 250 steps. Show how the evolution of density, temperature, and pressure compare using the different solvers at the 250th time step. Repeat the process for the different slope limiters and the exact Godunov solver. Zoom into various features as needed to show differences. The contents of `sod.dat` are as follows:

```
START "1 dimensional sod test"
    LEVELS 1
    XYZ 1
        X 200 -1.0 1.0 FREE
    SCALARS 2 ADVECT 2
    CASE SOD
END
$read x
march 250 plot rho p marker 4 p
```

10. Incorporate a cylindrical obstacle in the domain that occupies a blockage ratio of 0.75, and vary A to produce different outcomes. The geometry for this example can be generated via

```
solidbody
rad 15.0
```

11. Create a variable and a *command* called `x_cylinder` to control the x-location of where the center of the cylinder is placed in the domain. You will need to add it as a global variable to `user.h` (two places), and the following functions will need to be modified: `usrqis`, `usrqcmd`, `usrrd`, `usrwr`, `usrdef`. Find a good location for the cylinder that allows the wave speed to settle to a quasi-steady speed prior to interaction. Do this, and vary A, for both methane–oxygen and hydrogen–oxygen.

Need to add some example of using the LEM here

4 Detailed chemistry code (MG_CTI)

4.1 Governing Euler equations of motion

The complete set of conservation laws for mass, momentum, total energy, and ith chemical species solved here is

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \boldsymbol{u}) = 0 \tag{19}$$

$$\frac{\partial (\rho \boldsymbol{u})}{\partial t} + \nabla \cdot (\rho \boldsymbol{u} \otimes \boldsymbol{u}) + \nabla p = 0 \tag{20}$$

$$\frac{\partial (\rho E)}{\partial t} + \nabla \cdot \Big((\rho E + p) \boldsymbol{u} \Big) = 0 \tag{21}$$

$$\frac{\partial (\rho Y_i)}{\partial t} + \nabla \cdot (\rho \boldsymbol{u} Y_i) = \dot{\omega_i}, \tag{22}$$

where ρ, $\boldsymbol{u}$, p, Y_i, $\dot{\omega_i}$, refer to the density, velocity vector, pressure, mass fraction of the ith species, and the reaction rate of of the ith species, respectively. The total specific energy for a thermally perfect gas is given by

$$E = \sum (Y_i h_i) - \frac{p}{\rho} + \frac{1}{2} |\boldsymbol{u}^2|, \tag{23}$$

where h_i is the enthalpy of the ith species, and the temperature (T) is determined by the ideal gas law,

$$p = \rho R T, \tag{24}$$

where R is the specific gas constant. Finally, the speed of sound is computed using the chemically frozen ratio of specific heat capacities, $\gamma = c_p / c_v$, through

$$c^2 = \gamma p / \rho. \tag{25}$$

The specific heat capacities, c_p and c_v, and enthalpies for each species, h_i, are determined by the usual temperature dependent NASA polynomial approximations [1] for a multi-component gas.

4.2 The source terms

The general form of a system of I chemical reaction pathways, involving K chemical species, can be expressed in the form

$$\sum_{k=1}^{K} \nu'_{ki} \chi_k \rightleftharpoons \sum_{k=1}^{K} \nu''_{ki} \chi_k \qquad (i = 1, ..., I) \tag{26}$$

where ν'_{ki} and ν''_{ki} are the stoichiometric coefficients of the forward and backward reactions, respectively, and χ_k is the chemical symbol for the kth species. The

production rate of the kth species must account for the sum of all reactions involving such kth species, thus

$$\dot{\omega}_k = \sum_{i=1}^{I} (\nu''_{ki} - \nu'_{ki}) q_i \qquad (k = 1, ..., K) \tag{27}$$

where q_i is the rate of progress variable for the ith reaction given by the difference of forward and and reverse rates through

$$q_i = k_{fi} \prod_{k=1}^{K} [X_k]^{\nu'_{ki}} - k_{ri} \prod_{k=1}^{K} [X_k]^{\nu''_{ki}} \tag{28}$$

where $[X_k]$ is the molar concentration of the $k$tj species, and k_{fi} and k_{ri} are the specific forward and reverse reaction rate constants of the ith reaction.

The forward reaction rate constant for the I reactions are assumed to have the Arrhenius form

$$k_{fi} = A_i T^{\beta_i} \exp\left(\frac{-Ea_i}{RT}\right) \tag{29}$$

where A_i is the pre-exponential factor, β_i is the temperature exponent, Ea_i is the activation energy of the ith reaction, and R is the specific gas constant. Finally, the reverse reaction rate constant are related to the forward reaction rates through the equilibrium constants:

$$k_{ri} = k_{fi} / K_{ci} \tag{30}$$

where K_{ci} is the equilibrium constant of the ith reaction, and is given by

$$K_{ci} = K_{pi} \left(\frac{p_{\text{atm}}}{RT}\right)^{\sum_{k=1}^{K} \nu_{ki}} \tag{31}$$

and

$$K_{pi} = \exp\left(\frac{\Delta S_i^0}{R} - \frac{\Delta H_i^0}{RT}\right) \tag{32}$$

where

$$\frac{\Delta S_i^0}{R} = \sum_{k=1}^{K} (\nu''_{ki} - \nu'_{ki}) \frac{S_k^0}{R} \tag{33}$$

and

$$\frac{\Delta H_i^0}{RT} = \sum_{k=1}^{K} (\nu''_{ki} - \nu'_{ki}) \frac{H_k^0}{RT} \tag{34}$$

where S_k^0 and H_k^0 are the standard state molar entropy and molar enthalpy of the kth species, respectively. We note that for a spontaneous process, the change in Gibbs free energy from reactants to products is negative, or $\Delta G = \Delta H - T\Delta S < 0$, while at equilibrium, $\Delta G = 0$.

4.3 The basic script

The following basic script that models a detonation wave in methane-oxygen using detailed chemistry (see `detonation2D.dat`). Note, all units in MG_CTI are dimensional (SI).

```
! grid refinement settings
remhole
rtol 1E-3
abstol 1E-4
refcrit 0.05
derefcrit 0.04

! tolerance for ZND solver
ztol 1E-4

!solver settings
reactive
hllc
vanalbada
noviscous

! Read mechanisms and initial conditions
cti vijay2020                    ! specify cti file to read
! Need to specify upper temp limit of mechanism
! beyond this limit, nasa data will be extrapolated linearly
max_mech_T 5000.0
max_mech_T_db 5000.0     ! double precision version

! Initial temperature in K and pressure in Pa
T_initial 300
P_initial 3500.0
!rho_initial 1                   ! can alternatively specify density
! Initial mole fraction
$read XInitial_vijay2020                ! 16 species, 34 reactions
DCJin 2243.76                    ! Must determine beforehand
overdrive 1.1

! In this case, sootfoil will record max pressure
sootfoil

! load the model, disances in m
diffbad 100
START "2D CH4-O2 Detonation"
   LEVELS 4
   XYZ 2
       X 250 -0.2 9.8 SYMMETRIC-FREE
```

```
        Y 5 0.0 0.2 SYMMETRIC
    CASE CH4EXAMPLE
END
diffbad 20
```

By default, everything has been setup for you in the `usr.c` file for the `CH4EXAMPLE` case. You can search for "CH4EXAMPLE" in `usr.c` to see where specific custom directives for this case are included. This case calculates and sets up a ZND profile located at $x = 0$, overdriven slightly ($f = (D^2/D_{cj}^2) = 1.1$) to overcome startup errors at the beginning. A small zone of perturbations to the density field are located between $0 < x < 5$ mm. *Note:* In MG_CTI, it is no longer necessary to specify the number of scalars, as this is determined automatically.

For the example below, you can download the run scripts from the following bitbucket repository:

```
git clone git@bitbucket.org:bmaxw005/mg_trainingexample3.git
```

4.4 Training examples to try

Focus on H2 detonation example instead

1. Run the script for different resolutions, by increasing the number of AMR levels. Each AMR level effectively reduces the grid spacing in each direction by half. Compare (plot) the run times and numerical sootfoil images while keeping the number of CPUs constant. On the plot, make note of what coarsest resolution is required to produce *cells*. Report the resolutions in micrometers.

2. For one of the lowest resolution required to produce cells, start by running the script for 1 CPU and record the time needed for the simulation. Repeat the task over and over by doubling the number of CPUs. Thus, record and plot the simulation run time vs. n_{cpu}. Compare the cellular structure (i.e. numerical sootfoil images) for each simulation. By analysing the `mgout` file generated and run times, estimate roughly how many cells per CPU is optimal.

3. For an intermediate resolution (not too high, not too low), and pick a reasonable number of CPUs to use such that computations can be done in a reasonable time. Now, modify the main solution tolerances and grid refinement parameters, `rtol`, `abstol`, `diffbad`, `refcrit`, and `derefcrit`. How do changes in each of these parameters change the grid topology and the resulting sootfoil image? Explain what each of these parameters does. *Hint:* Try to decipher how these parameters control the refinement in the `usrqsol` function in `usr.c`. *Note:* In this case, `rtol` and `abstol` control only the AMR refinement. Also, very specific criteria needs to be prescribed for refinement on each species, and this likely has to be determined on a case-by-case basis.

4. Pick a reasonable number of CPUs to use, satisfactory grid refinement strategy, and resolution from the previous tasks. Compare your solution to that obtained for a comparable resolution using the one- and four-step combustion models.

5. Incorporate a cylindrical obstacle in the domain that occupies a blockage ratio of 0.75, and vary the initial pressure, p_0, to produce different outcomes. Find a good location for the cylinder that allows the wave speed to settle to a quasi-steady speed prior to interaction. Compare your solution outcomes to those obtained using the one-step model. The geometry for this example can be generated via

   ```
   solidbody
   radius 0.15
   x_position 5.0
   y_position 0.0
   ```

   Note: Unlike MG_LEM, the internal geometry is not stored as a solution variable. Instead, cells are flagged using an interger value, i.e. `cptr->solid`, which is set to 0 for a fluid, 1 for partially solid, and 2 for completely solid. The boundaries of the internal geometries are set up in the `usrCheckSolid` function.

   Note: The `autoref` command can be invoked which disables refinement around the internal geometry while the fluid is at rest. This is intended to speed up computations a little by only refining around the object as the wave dynamics approach, however it may not be as stable and requires more testing.

added by Samaneh:

5 CFD workflow - Fundamentals

CFD codes follow this fundamental workflow: Pre-processing, Solving, and Post-processing. Here is a simple breakdown for a CFD simulation:

5.1 Stage 1: Pre-processing

This is where you define the physical problem and translate it into a format the computer can understand.

- **Geometry definition:** You must define the physical boundaries of the flow field.
- **Grid generation:** The fluid domain is discretized into a number of small cells. The grid resolution is critical; you need a finer mesh where high gradients occur, such as across a shock wave or a flame front.

- **Physical modeling:** You determine which governing equations to use. This involves selecting Euler or Navier-Stokes equations, and deciding on source terms for turbulence, radiation, or chemical reactions (e.g., a one-step reaction model vs. detailed chemistry). A proper closure model may need to be determined here.
- **Initial conditions:** You must define the initial state of the fluid. For example, in a shock-tube problem where a diaphragm divides high-pressure and low-pressure gases, you must specify the initial state at time zero, including the pressure, density, velocity, etc. of both gases.
- **Boundary conditions:** You also need to determine the conditions at the boundaries (e.g., solid walls, symmetrical boundaries, pressure outlets, free boundaries).

5.2 Stage 2: Solving

Now that the problem is set up, you can start computing the solution.

- **Discretization of equations:** The governing fluid equations exist in continuous forms–continuous partial differential equations. Using the finite volume method, you discretize this system of equations.

To see how this works, let's look at a specific example. In 1D, inviscid fluid flow (ignore external source terms), the conservation of momentum is expressed as:

$$\frac{\partial(\rho u)}{\partial t} + \frac{\partial(\rho u^2 + p)}{\partial x} = 0$$

Where:

- ρ: Fluid density
- u: Fluid velocity in x-direction
- p: Static pressure
- t: Time
- x: Spatial coordinate

The first term, $\frac{\partial(\rho u)}{\partial t}$, represents the rate of change of momentum. The second term, $\frac{\partial(\rho u^2+p)}{\partial x}$, represents the spatial gradient of the momentum flux.

To solve this on a computer, the continuous equation must be broken down across a grid. Using an explicit finite volume method, we divide the 1D domain into cells of width Δx and advance the solution by a time step Δt. Subscript i denotes the current cell center, and superscripts n and $n+1$ denote the current and future time steps, respectively. The discretized form is:

$$\frac{(\rho u)_i^{n+1} - (\rho u)_i^n}{\Delta t} + \frac{F_{i+1/2}^n - F_{i-1/2}^n}{\Delta x} = 0$$

Rearranging this to solve for the momentum, ρu, inside the cell at the next time step $(n+1)$, we get the algebraic equation that the CFD solver actually computes:

$$(\rho u)_i^{n+1} = (\rho u)_i^n - \frac{\Delta t}{\Delta x}\left(F_{i+1/2}^n - F_{i-1/2}^n\right)$$

Where the flux terms (F) at the right face $(i+1/2)$ and left face $(i-1/2)$ of the cell are $F = (\rho u^2 + p)$.

Calculating the exact values of those fluxes at the cell boundaries rather than the cell centers requires utilizing a proper numerical scheme. This is exactly where specific numerical schemes, like Godunov, HLL, or Roe solvers, are used.

Note: Godunov's method provides the Riemann-problem framework for computing fluxes. First-order Godunov uses piecewise-constant reconstruction, while second-order Godunov uses MUSCL reconstruction with slope limiters, (van Albada, Minmod, Superbee, van Leer) to achieve higher spatial accuracy.

Time accuracy is obtained independently, commonly using predictor–corrector schemes, which provides second-order accuracy in time.

- **Spatial and temporal integration:** The solver calculates how properties like mass, momentum, energy, and species transport change from one cell to the next (spatial) and from one time-step to the next (temporal).
- **Iterative calculations:** At each time step, the solver iterates through the system of equations repeatedly until the solution "converges"—meaning the error (residuals) drops below a defined tolerance (check `rtol` as a relevant tolerance used in the code).
- **Adaptive Mesh Refinement (AMR):** The solver will dynamically refine or coarsen the grid during the calculation to follow moving features, ensuring computational efficiency (`refcrit` and `derefcrit` are the relevant tolerances here).

5.3 Stage 3: Post-processing

Now that the calculation is complete, you can extract physical meaning from the raw numerical data.

- **Visualization:** This involves plotting the data so you can actually see the flow field evolution. You can generate contour plots of temperature, pressure, and density, or plot numerical soot-foil records (to illustrate the cellular structure of a detonation wave).
- **Other:** You can extract specific quantities, such as the flame or shock speed, or track the changes of a specific property at a given x over time, or track the changes of a specific property at a given time over x, etc.

6 MG Code Files

Each MG directory contains multiple files. This modular approach allows for a more manageable workflow, rather than implementing all the functions and algorithms into a single self-contained script.

6.1 1. Data structures and definitions

These header files define what data exists and how it is organized in memory.

- `struct.h`: It defines the foundational data structures used across the entire program. This includes definitions for the mesh (nodes, cells, structures for AMR), state variables (density, momentum, energy), geometric boundaries, etc.
- `user.h`: A customizable header file where specific global variables and model constants (such as the radius and coordinates of a cylindrical obstacle) are declared for your specific simulation case.

6.2 2. Initialization, setup, execution, and memory allocation

These files manage how the simulation operates from the moment it is launched until it finishes execution.

- `mg.c`: This contains the `main()` function. It initializes the system, parses the input scripts (`.dat` scripts), and dictates when to march forward in time, update the grid, and write outputs.
- `start.c`: It allocates memory for the grid, sets up the initial computational domain, and prepares the environment before the first time-step begins.

6.3 3. Numerical solvers and integration

These files contain the numerical solvers that actually solve the governing fluid equations (Euler/Navier-Stokes).

- `march.c`: It contains the functions that advance the fluid state from time t to $t + \Delta t$. It calls the flux solvers (e.g., Godunov, HLL, and Roe solvers) and applies the slope limiters to update the variables in each cell.
- `adi.c`: This file is used when implicit time-stepping is applied instead of explicit methods.

6.4 4. Mesh management

- grid.c: Handles the Adaptive Mesh Refinement (AMR) topology. When the simulation detects sharp gradients (such as across a shock wave), it divides cells (refines) or merges them back together (derefines). It also manages ghost cells and cells located at the boundaries.

6.5 5. Parallelization and I/O

- para.c: Manages parallel computing. It handles domain decomposition (splitting the grid among multiple CPUs) and the necessary communication between adjacent grid blocks on different processors.
- modelrw.c: Handles Input/Output (Read/Write) operations. It is responsible for writing simulation data to disk (e.g., plot files) and reading them back in if you need to restart a simulation from a saved state.

6.6 6. User customization

- usr.c: A personalized file, which noticeably differs from one MG code to another (e.g., mg_g, mg_cti), where you define your specific problem. It contains functions to set custom initial conditions, add source terms (such as chemical reactions), and define custom grid refinement criteria.

6.7 How are they linked together?

1. You run the executable (e.g., using the command mg_cti in your terminal). mg.c starts and includes struct.h and user.h to know what data exists.
2. mg.c calls functions in start.c to allocate memory and calls functions in usr.c to apply your specific initial conditions and parameters.
3. mg.c enters a loop:
   - It calls march.c to calculate fluxes and update the physical state (mass, momentum, energy).
   - If implicit methods are used, march.c calls adi.c.
   - para.c works behind the scenes to sync boundaries across CPUs.
4. Once the simulation advances, mg.c calls grid.c to check if the mesh needs to be refined to capture sharp-gradient features (like a detonation wave or shock). It uses tolerances defined in usr.c to make this decision.
5. Periodically, mg.c calls modelrw.c to save the current state to your hard drive.

References

[1] Robert J Kee, Fran M Rupley, Ellen Meeks, and James A Miller. CHEMKIN-III: A FORTRAN chemical kinetics package for the analysis of gas-phase chemical and plasma kinetics. Technical report, Sandia National Labs., Livermore, CA (United States), 1996.